ShadowDZ at HalluScoring-2026: A Fine-Tuned Fanar Approach for Islamic Knowledge Hallucination Detection

Faiz Kraimia
Faculty of New Information
and Communication Technologies
University of Constantine 2
Abdelhamid Mehri
Constantine 25000, Algeria
faiz.kraimia@univ-constantine2.dz

Aicha Eutamene
Faculty of Natural
and Life Sciences
University of Constantine 1
Freres Mentouri
Constantine 25000, Algeria
aicha.eutamene@umc.edu.dz

Abstract

Large language models used for question answering can produce fluent but factually incorrect answers, a problem that is acute in knowledge-sensitive domains such as Islamic religious knowledge. We describe our system for the HalluScoring 2026 shared task (Islamic Knowledge track), which requires jointly detecting hallucinated answers and selecting the correct option among six candidates. We fine-tune Fanar-1-9B-Instruct with LoRA and decompose inference into two independently-prompted steps, judging hallucination before revealing the candidate options, to prevent a textual-similarity shortcut where one option is a near-verbatim restatement of the generated answer. Our system scores 0.7149 (Assisted) and 0.6222 (Blind) on the official hidden test set, ranking 5th of 5 reported teams; a controlled comparison shows this shortcut-resistant design costs roughly 10 points against a single-pass variant on a held-out development subsample (0.7625 vs. 0.8639 Combined Score). We release our code and model to support further work on this trade-off.

1 Introduction

Large Language Models (LLMs) have demonstrated strong capabilities in answering questions across diverse domains, yet they remain prone to hallucination, generating fluent and confident text that is factually incorrect, unsupported, or misleading (Ji et al., 2023). This issue is particularly critical in knowledge-sensitive domains such as Islamic religious knowledge, where factual errors involving Qur'anic references, Hadith attributions, or jurisprudential rulings can have significant real-world consequences. This work addresses hallucination detection and correction for **Modern Standard Arabic** questions in the Islamic knowledge domain.

The HalluScoring 2026 shared task (Alansari et al., 2026), built upon the HalluTruthQA benchmark (Bouchekif et al., 2026b; Bekhouche et al., 2026), frames this problem as a unified two-step task: given a question, an LLM-generated answer, and six candidate options, participating systems must (1) determine whether the generated answer contains hallucinated information, and (2) select the factually correct option among the six candidates. This paper describes our system, submitted by team ShadowDZ, for the Islamic Knowledge track of this shared task.[1]

Our main strategy is to fine-tune an open-source generative language model, Fanar-1-9B-Instruct (Team et al., 2025), using Low-Rank Adaptation (LoRA) (Hu et al., 2021). Rather than using a single combined prompt, we split inference into two independent, sequentially-prompted steps, so the model cannot infer the hallucination label from textual similarity to the candidate options instead of genuine factual verification. The full rationale is given in Section 3.

On the official hidden test set, our system achieves an Assisted Score of 0.7149 and a Blind Score of 0.6222, ranking 5th among five reported teams on the Islamic Knowledge track. The main challenge we encountered was a measurable trade-off between this stricter, shortcut-resistant inference design and raw leaderboard performance: on a genuinely held-out development subset, our two-step architecture scored 0.7625, noticeably lower than a single-pass variant we also developed (0.8639), which we hypothesize benefits from the aforementioned textual-similarity shortcut.

2 Background

Task setup. Each instance provides a question, an answer generated by a baseline LLM, and six candidate answers labeled A–F. For example, given the question من أول من بدأ بجمع القرآن الكريم من الخلفاء الراشدين؟

[1] Code available at https://github.com/KraimiaFaiz/halluscoring-2026-islamic.

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1176–1181
October 28-29, 2026 ©2026 Association for Computational Linguistics

(*Which of the Rightly-Guided Caliphs first undertook the compilation of the Qur'an?*) and a generated answer naming Abū Bakr, systems must output a binary hallucination label (`hallucination` / `no_hallucination`) and select the option (A–F) that best answers the question. By construction, this coincides with the generated answer when it is labeled `no_hallucination`, and with the correct factual answer otherwise.

Dataset. We use the Islamic Knowledge subset of HalluTruthQA-4K (Bekhouche et al., 2026), the official dataset for Track 2 of HalluScoring 2026, itself an extension of the original HalluTruthQA benchmark (Bouchekif et al., 2026b). All questions and answers are in Modern Standard Arabic, covering Qur'anic interpretation, Hadith, Islamic jurisprudence (fiqh), and Islamic history. The Islamic Knowledge track provides 400 training, 200 development, and 400 test instances (1,000 total); gold labels and answers are released for the training and development splits only.

Track. We participate in a single track: Islamic Knowledge (Task 2.1).

Related work. Hallucination in LLM-generated text has been studied extensively in English (Ji et al., 2023). Work on Arabic remains comparatively limited, motivating benchmarks such as HalluTruthQA (Bouchekif et al., 2026b; Bekhouche et al., 2026) and related work on configuring deep architectures for Arabic (Saadi and Belhadef, 2018, 2020). Among concurrent submissions to the same track, Ziani (2026) fine-tune `google/gemma-4-12B-it` with deterministic decoding, normalizing raw generations to extract the label and option. Unlike this and other single-pass formulations, we investigate whether withholding candidate options during hallucination judgment reduces a model's reliance on textual-similarity shortcuts, a question not explicitly addressed in concurrent work on this task. Additional related work is discussed in Appendix A.

3 System Overview

Base model and fine-tuning. We fine-tune Fanar-1-9B-Instruct (Team et al., 2025) (Fanar Prime, 8.7B parameters, continually pretrained from Gemma-2 9B on Arabic, English, and code data) with LoRA (Hu et al., 2021) (rank r=16, α=32, dropout 0.05), applied to all attention and MLP projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`). The base model is loaded in 4-bit NF4 quantization during training. Training uses a per-device batch size of 1 with gradient accumulation over 16 steps (effective batch size 16), a learning rate of 2×10^{-4} with cosine decay, and 2 epochs.

Two-stage training. We fine-tune in two stages: *Stage A* trains on `train` only, re-initializing LoRA weights for *Stage B*, which trains on `train+dev` combined and produces the official submission checkpoint (data splits detailed in Section 4). Because Stage B sees all released labels, its performance on `dev` reflects memorization rather than generalization; we rely on Stage A for an unbiased estimate (Section 5).

Two-step inference (main design decision). Rather than presenting the six candidate options alongside the generated answer in a single prompt, we decompose inference into two sequential, independently-prompted steps, illustrated in Figure 1 and summarized in Algorithm 1. This addresses the following task challenge: because one candidate option is always a near-verbatim restatement of the generated answer, exposing all options during hallucination judgment allows a model to infer the label via textual similarity rather than independent verification of the answer's factual content.

Algorithm 1 Two-step inference

Require: question q, generated answer a, options $\{o_A, \ldots, o_F\}$
1: $v \leftarrow \textsc{Generate}(\texttt{prompt}_1(q, a))$ ▷ Step 1: no options shown
2: $\hat{y} \leftarrow \textsc{ParseLabel}(v) \in \{\texttt{hallucination}, \texttt{no_hallucination}\}$
3: $g \leftarrow \textsc{Generate}(\texttt{prompt}_2(q, a, \hat{y}, \{o_A, \ldots, o_F\}))$ ▷ Step 2: informed by $\hat{y}$
4: $\hat{o} \leftarrow \textsc{ParseOption}(g) \in \{A, \ldots, F\}$
5: **return** $(\hat{y}, \hat{o})$

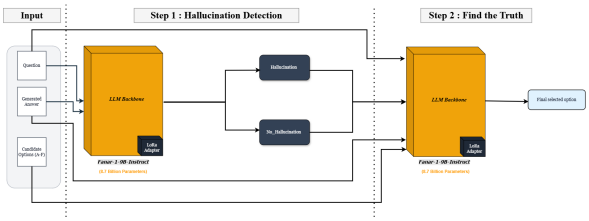


Figure 1: Two-step inference architecture.

Concrete example. A full worked example is given in Appendix B.

Decoding and parsing. Both steps use greedy decoding (full parameters in Appendix D). During Stage B training, the ground-truth label is used as $\hat{y}$ in Step 2's prompt (teacher forcing); at inference, the model's own Step 1 prediction is used, so an incorrect Step 1 verdict can propagate to Step 2.

Single-pass configuration (comparison system). For comparison (Section 5), we additionally train a single-pass configuration with the same base model, LoRA configuration, and training procedure, but where hallucination judgment and option selection are produced in one generation from a single prompt containing the question, generated answer, and all six options simultaneously. Because one option is always a near-verbatim restatement of the generated answer, this setup lets the model infer the hallucination label by matching the generated answer's wording against the options directly, rather than verifying its factual content. This textual match is not a valid signal of factual correctness, since a fluently-worded hallucination could still surface a similar option, which our two-step design is intended to avoid.

4 Experimental Setup

Data splits. The track provides 400 `train`, 200 `dev`, and 400 `test` instances. Stage A is evaluated on a random 30-instance `dev` subsample (seed 42) held out from training. Stage B trains on `train+dev` (600 instances) and is evaluated on the hidden `test` split; a further 20-instance `dev` subsample serves as a post-training sanity check only (not a generalization estimate).

Preprocessing. Each training instance is converted into two supervised examples, one per inference step (Section 3): a Step 1 example (prompt without options, target is the analysis and hallucination label) and a Step 2 example (prompt with the gold label and all six options, target is the analysis and correct letter). Prompts are formatted with the base model's chat template and truncated to 1536 tokens. Loss is computed only on the completion following the prompt, using a completion-only data collator that masks the prompt tokens.

Hyperparameters. We fine-tune with LoRA (r=16, α=32) under 4-bit NF4 quantization for 2 epochs (learning rate 2×10^{-4}, effective batch size 16); inference uses greedy decoding. The complete configuration, sufficient for replication, is listed in Appendix D.

Software. We use Hugging Face `transformers` (4.44–4.50), `peft` (0.11–0.14), `trl` (0.11–0.13), `accelerate` ($\geq$0.31), `bitsandbytes` ($\geq$0.43), and `datasets` ($\geq$2.19)[2], together with PyTorch and scikit-learn for evaluation. No additional external datasets, retrieval systems, or proprietary APIs are used.

Evaluation metrics. Following the shared task's official formula, systems are scored with a Combined Score $= 0.5\times$ Macro F1-Score (`hallucination/no_hallucination`, macro-averaged) $+ 0.5\times$ Option Accuracy (exact match on the selected letter). The leaderboard reports this as the *Assisted Score*, alongside a complementary *Blind Score* discussed in Section 5.

5 Results

5.1 Official results

Table 1 shows the official leaderboard for the Islamic Knowledge track, on the hidden `test` split (400 instances), released after submission. Our two-step system ranks 5th out of 5 reported teams, with an Assisted Score of 0.7149 and a Blind Score of 0.6222.

Team	Assisted	Blind
Anupamhashtag	0.8124	0.8262
CH-A-Malek	0.7881	0.7582
IslamicAI	0.7313	0.6541
Yazid_Bounab	0.7199	0.6661
ShadowDZ (ours)	**0.7149**	**0.6222**

Table 1: Official test-set leaderboard, Islamic Knowledge track.

Based on the organizers' evaluation script, we hypothesize that Blind Score reflects hallucination-detection quality in isolation, while Assisted Score reflects the combined task; we could not confirm this with the organizers before submission. A separate, non-comparable Development-phase score is reported in Appendix C for completeness.

5.2 Design decision impact (post-submission analysis)

To quantify the cost of our two-step design, we compare it against the single-pass configuration described in Section 3, both trained identically on `train` only (Stage A) and evaluated on the same held-out subsample of 30 `dev` instances never seen

[2] URLs: github.com/huggingface/transformers, github.com/huggingface/peft, github.com/huggingface/trl, github.com/huggingface/accelerate, github.com/bitsandbytes-foundation/bitsandbytes, github.com/huggingface/datasets

during training. This comparison was conducted after official submission and is not part of the leaderboard evaluation.

Config.	Macro F1	Opt. Acc	Combined
Single-pass	0.8611	0.8667	0.8639
Two-step (ours)	0.7917	0.7333	0.7625

Table 2: Held-out dev subsample (30 instances, Stage A, never used for training either configuration).

The single-pass configuration scores roughly 10 points higher on this held-out split. We attribute this gap to the textual-similarity shortcut described in Section 1: with all options visible during hallucination judgment, the single-pass model can align the generated answer against the options directly, whereas our two-step model must judge hallucination from the question and generated answer alone, a strictly harder sub-problem. We note this same single-pass configuration was never submitted for official test-set evaluation, so this comparison is necessarily confined to the held-out dev subsample.

5.3 Error analysis

On this held-out subsample, the single-pass configuration's confusion matrix is as follows: for the `hallucination` class (support 14), precision 1.00 and recall 0.71; for `no_hallucination` (support 16), precision 0.80 and recall 1.00. All errors are false negatives on `hallucination` instances that the model reclassifies as `no_hallucination`. This is consistent with the shortcut hypothesis, since a model matching the generated answer to a superficially similar option would default to `no_hallucination`.

All three option-selection errors in this run share the same gold answer, option C, while the model instead predicted E, B, and F, respectively (e.g., for instance `islamicHallucination_747`, gold `C`, predicted `E`). In each case, the model's justification states the generated answer is مطابقة تمامًا للمعلومة الصحيحة (*exactly matching the correct information*), indicating the model defaulted to endorsing the generated answer rather than identifying the specific factual error. This is precisely the shortcut behavior our two-step design targets. We do not have an equivalent per-instance error breakdown for the two-step configuration, as its evaluation logs retained only aggregate metrics.

6 Conclusion

Summary. We presented a two-step inference pipeline for the HalluScoring 2026 Islamic Knowledge track, fine-tuning Fanar-1-9B-Instruct with LoRA to judge hallucination and select the correct answer as two independently-prompted steps, rather than a single combined prompt. This design is specifically intended to prevent reliance on a textual-similarity shortcut available when candidate options are shown alongside the generated answer during hallucination judgment. Our system scored 0.7149 (Assisted) and 0.6222 (Blind) on the official hidden test set, ranking 5th of 5 reported teams. On a genuinely held-out development subsample, our two-step configuration scored 0.7625, compared to 0.8639 for an otherwise identical single-pass configuration, quantifying the cost of removing this shortcut.

Limitations. The controlled comparison in Table 2 isolates the design choice but is confined to a small held-out subsample (30 instances), a single base model, and a single seed. Our architecture also introduces an error-propagation risk absent by construction in single-pass systems, since an incorrect Step 1 verdict can mislead Step 2's judgment, and compute constraints limited us to 2 epochs without a systematic hyperparameter search. Finally, we only evaluate on the Islamic Knowledge track; generalization to the General Culture track or other languages remains untested.

Future work. Promising directions include mitigating Step 1-to-Step 2 error propagation, for example through conditioning Step 2 on a confidence estimate or joint training of the two steps, and running a systematic hyperparameter search across seeds. We also plan to extend the design to the General Culture track and other base models, and to explore reinforcement-learning-based training (Ouamane and Belhadef, 2022) to directly optimize the Combined Score.

Acknowledgments

We thank Hacene Belhadef for helpful discussions and advice, and the HalluScoring 2026 organizing team for maintaining the shared task and evaluation infrastructure.

References

Aisha Alansari, Abdessalam Bouchekif, Ahmed Hasanaath, Salah Eddine Bekhouche, Malak Alkhorasani, Mohammed-En-Nadhir Zighem, Saad Ezzini, Hichem Telli, Hend Al-Khalifa, Muhammad Abdul-Mageed, Abdenour Hadid, and Hamzah Luqman. 2026. Halluscoring 2026: The first shared task on llms hallucination detection and answer verification. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Salah Eddine Bekhouche, Abdessalam Bouchekif, Hichem Telli, Mohammed-En-Nadhir Zighem, and Abdenour Hadid. 2026. Hallutruthqa-4k: A fine-grained corpus and annotation process for arabic hallucination detection and truth verification. *arXiv preprint arXiv:2608.03966*.

Abdessalam Bouchekif, Shahd Gaben, Samer Rashwani, Somaya Eltanbouly, Mutaz Al-Khatib, Heba Sbahi, Mohammed Ghaly, and Emad Mohamed. 2026a. Mawarith: A dataset and benchmark for legal inheritance reasoning with llms. *arXiv preprint arXiv:2603.07539*.

Abdessalam Bouchekif, Samer Rashwani, Heba Sbahi, Shahd Gaben, Mutaz Al Khatib, and Mohammed Ghaly. 2025. Assessing large language models on islamic legal reasoning: Evidence from inheritance law evaluation. In *Proceedings of The Third Arabic Natural Language Processing Conference*, pages 246–257.

Abdessalam Bouchekif, Mohammed-En-Nadhir Zighem, Salah Eddine Bekhouche, Hichem Telli, Somaya Eltanbouly, Shahd Gaben, Heba Sbahi, Samer Rashwani, Mutaz Al-Khatib, Emad Mohamed, et al. 2026b. Hallutruthqa: A fine-grained benchmark for hallucination detection, localization, and explanation in arabic question answering. *arXiv preprint arXiv:2607.20219*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38.

Nour El Houda Ouamane and Hacene Belhadef. 2022. Deep reinforcement learning applied to nlp: a brief survey. In *2022 2nd International Conference on New Technologies of Information and Communication (NTIC)*, pages 1–5. IEEE.

Abdelhalim Saadi and Hacene Belhadef. 2018. A statistical approach for the best deep neural network configuration for arabic language processing. In *International Symposium on Modelling and Implementation of Complex Systems*, pages 204–218. Springer.

Abdelhalim Saadi and Hacene Belhadef. 2020. Deep neural networks for arabic information extraction. *Smart and Sustainable Built Environment*, 9(4):467–482.

Fanar Team, Ummar Abbas, Mohammad Shahmeer Ahmad, Firoj Alam, Enes Altinisik, Ehsannedin Asgari, Yazan Boshmaf, Sabri Boughorbel, Sanjay Chawla, Shammur Chowdhury, et al. 2025. Fanar: An arabic-centric multimodal generative ai platform. *arXiv preprint arXiv:2501.13944*.

Khaled Ziani. 2026. Detecting hallucinations and recovering verified answers in arabic islamic question answering. *arXiv preprint arXiv:2608.03720*.

A Additional Related Work

A separate line of work evaluates LLMs directly on Islamic legal reasoning, closely related to the factual correctness problem addressed here. Bouchekif et al. (2025) benchmark seven LLMs on 1,000 multiple-choice questions covering Islamic inheritance law (*'ilm al-mawārīth*), reporting a wide performance gap: models such as o3 and Gemini 2.5 exceed 90% accuracy, while others, including Fanar, the base model used in this work, score below 50%.

Bouchekif et al. (2026a) introduce MAWARITH, a larger benchmark of 12,500 Arabic inheritance cases annotated with step-by-step reasoning traces, enabling fine-grained evaluation of intermediate legal reasoning rather than final-answer accuracy alone. This structured, multi-stage evaluation style is conceptually related to our own two-step decomposition (Section 3), and suggests inheritance law as a promising domain for extending this work.

B Worked Example

Consider again the example from Section 2: the question "من أول من بدأ بجمع القرآن الكريم من الخلفاء الراشدين؟" with a generated answer naming Abū Bakr. In Step 1, the model receives only the question and this generated answer (no options) and outputs `no_hallucination`, since Abū Bakr was historically the first to compile the Qur'an into a single *muṣḥaf*. In Step 2, the model receives the question, the generated answer, the verdict $\hat{y} =$ `no_hallucination`, and the six options, one of which (option E) restates "Abū Bakr al-Ṣiddīq", and selects $\hat{o} =$ `E`, matching the generated

answer as expected under the task's decision rule (Section 2).

C Development-Phase Leaderboard

Separately from the final test-phase ranking reported in Section 5, an earlier Development-phase leaderboard scored predictions on the released dev split. Our single-pass system, trained with dev included in its training data (Stage B), scored 0.9177 there, ranking 2nd. We report this only for completeness: since dev labels were seen during training, this score reflects memorization rather than generalization, and is not comparable to the held-out results in Table 2 or the official test results in Table 1.

D Full Hyperparameter Configuration

Table 3 lists the complete configuration used for both training stages (Stage A and Stage B use identical hyperparameters, differing only in training data) and for inference, sufficient for replication.

Component	**Value**
Quantization	
Method	4-bit NF4
Double quantization	Yes
Compute dtype	float16
LoRA	
Rank (r)	16
Alpha (α)	32
Dropout	0.05
Target modules	q,k,v,o_proj, gate,up,down_proj
Training	
Per-device batch size	1
Gradient accumulation steps	16
Effective batch size	16
Learning rate	2×10^{-4}
LR scheduler	Cosine
Warmup ratio	0.03
Epochs	2
Optimizer	paged_adamw_8bit
Precision	fp16
Max sequence length	1536 tokens
Random seed	42
Inference	
Decoding strategy	Greedy
Max new tokens (Step 1)	60
Max new tokens (Step 2)	100

Table 3: Complete hyperparameter configuration.